

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN
LICENCIATURA EN INGENIERÍA EN SISTEMAS DE INFORMACIÓN
CURSO: Algoritmos
CATEDRÁTICO: Ing. Miguel Catalán



TEMA

Manual Técnico

NOMBRE DEL ALUMNO: Jonathan Esteven Barrera Morales
Carne No.: 7590-25-28376

SAN JUAN SACATEPÉQUEZ, 17-10-2025.

Manual Técnico - Gestor de Notas Académicas

Tabla de Contenidos

1. Descripción General
2. Arquitectura del Sistema
3. Estructuras de Datos
4. Módulos y Funciones
5. Algoritmos Implementados
6. Flujo de Datos
7. Análisis de Complejidad
8. Configuración e Instalación
9. Mantenimiento y Depuración

Descripción General

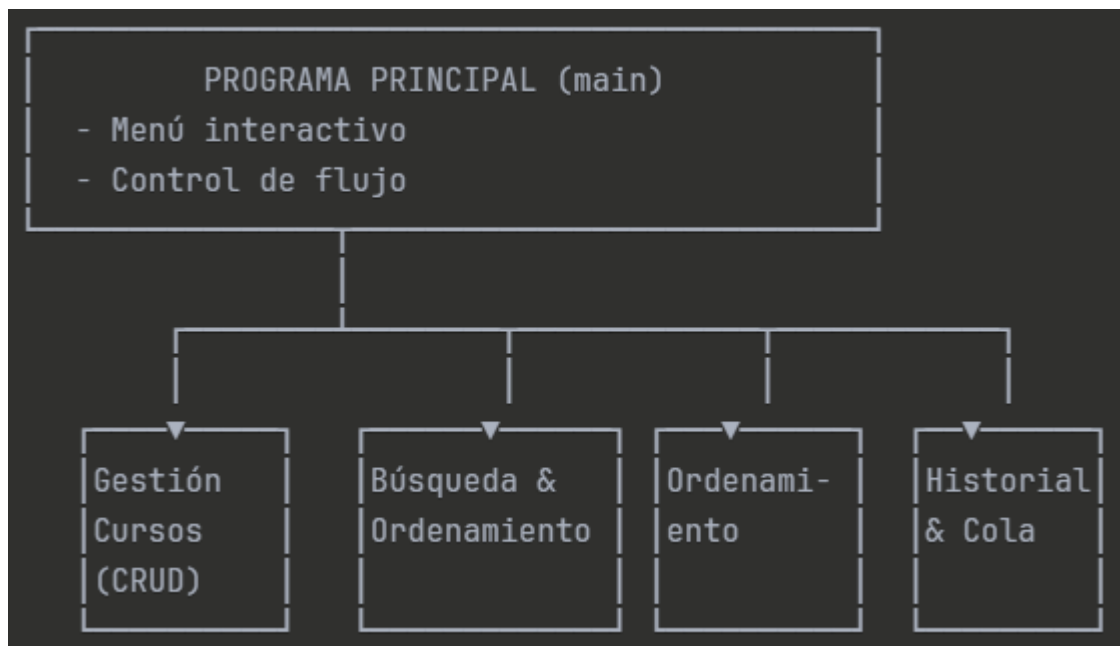
El **Gestor de Notas Académicas** es una aplicación Python que implementa conceptos fundamentales de programación incluyendo algoritmos de búsqueda y ordenamiento, estructuras de datos (pilas, colas, listas), y gestión de interfaz de usuario.

Características principales:

- Gestión CRUD de cursos
- Búsqueda lineal y binaria
- Ordenamiento por burbuja e inserción
- Cola de solicitudes (FIFO)
- Historial de cambios (LIFO)
- Cálculos estadísticos

Lenguaje: Python 3.7+ **Dependencias:** collections (módulo estándar)

Arquitectura del Sistema



Estructura General del Código

```
# 1. IMPORTACIONES
from collections import deque

# 2. MENÚ Y UI
def Mostrarmenu()

# 3. FUNCIONES CRUD
def pedir_curso()
def Mostrar_curso()
def eliminar_curso()

# 4. CÁLCULOS
def Promedio()
def aprobados_reprobado()

# 5. BÚSQUEDA
def busquedacursolineal()
def busqueda_binaria()

# 6. ORDENAMIENTO
def ordenar_nota()
def ordenar_curso()

# 7. HISTORIAL
def registrar_cambio()
def mostrar_historial()

# 8. COLA DE REVISIONES
def agregar_solicitud_revision()
def procesar_solicitud_revision()

# 9. PROGRAMA PRINCIPAL
cursos = []
cola_revisiones = deque()
historial_cambios = []
while repetir:
    # Menú y opciones
```

Estructuras de Datos

1. Lista de Cursos

```
cursos = [  
    {"nombre": "Programación", "nota": 85.5},  
    {"nombre": "Bases de Datos", "nota": 92.0},  
    ...  
]
```

Tipo: list de dict **Acceso:** $O(n)$ **Inserción:** $O(1)$ al final **Búsqueda:** $O(n)$

2. Cola de Revisiones

```
cola_revisiones = deque()  
# Estructura de cada elemento:  
{  
    'id': 1,  
    'curso': 'Programación',  
    'nota_actual': 85.5,  
    'estudiante': 'Juan Pérez',  
    'motivo': 'Creo que debería tener más puntos',  
    'estado': 'Pendiente'  
}
```

Tipo: deque (double-ended queue) **Operación:** FIFO (First In, First Out) **append():** Agregar al final - $O(1)$
popleft(): Remover del inicio - $O(1)$

```
historial_cambios = [  
    {  
        'tipo': 'REGISTRO DE CURSO',  
        'detalles': "Curso 'Programación' registrado con nota 85.50",  
        'fecha': "14/10/2025 14:30:45"  
    },  
    ...  
]
```

Tipo: list de dict **Operación:** LIFO (Last In, First Out - simulado) **Inserción:** $O(1)$ con append() **Acceso:** $O(n)$ recorrido inverso

Módulos y Funciones

Módulo 1: Menú Principal

python

```
def Mostrarmenu()
```

Propósito: Mostrar menú interactivo **Entrada:** Ninguna **Salida:** Imprime 13 opciones **Complejidad:** O(1)

Módulo 2: Gestión de Cursos (CRUD)

Crear

python

```
def pedir_curso()
```

- **Entrada:** Entrada del usuario (nombre, nota)
- **Validación:**
 - Nota entre 0-100
 - Nombre no vacío
- **Salida:** Diccionario {"nombre": str, "nota": float}
- **Complejidad:** O(1)

Leer

python

```
def Mostrar_curso()
```

- **Entrada:** Lista global cursos
- **Salida:** Tabla formateada en consola
- **Complejidad:** O(n)
- **Formato:**

#	Curso	Nota	Estado
1	Prog	85.50	APROBADO

Actualizar

python

```
def actualizar_nota_curso(cursos, nombre_cursos, nota_nueva)
```

- **Entrada:** lista, nombre curso, nueva nota

- **Salida:** True si actualiza, False si no encuentra
- **Complejidad:** $O(n)$
- **Cambio:** Modifica cursos[i]['nota'] in-place

Eliminar

python

```
def eliminar_curso(cursos, remover)
```

- **Entrada:** lista de cursos, nombre a eliminar
- **Salida:** True si elimina, False si no encuentra
- **Complejidad:** $O(n)$
- **Operación:** cursos.remove(curso)

Módulo 3: Cálculos Estadísticos

Promedio

python

```
def Promedio()
```

Fórmula: promedio = sum(notas) / len(notas) **Complejidad:** $O(n)$ **Criterio aprobación:** ≥ 61

Conteo

python

```
def aprobados_reprobado()
```

Complejidad: $O(n)$ **Itera:** Cada curso y clasifica por nota **Umbral:** 61 puntos

Módulo 4: Búsqueda

Búsqueda Lineal

python

```
def busquedacursolineal(cursos, busqueda)
```

Algoritmo:

PARA cada curso EN cursos:

 SI curso.nombre.lower() == busqueda.lower():

 RETORNAR True

RETORNAR False

Complejidad: $O(n)$ **Mejor caso:** $O(1)$ - primer elemento **Peor caso:** $O(n)$ - último elemento o no existe
Ventaja: Funciona con lista desordenada **Desventaja:** Lenta con muchos datos

Búsqueda Binaria

python

```
def busqueda_binaria(cursos, busqueda)
```

Requisito: Lista **DEBE** estar ordenada alfabéticamente

Algoritmo:

```
izquierda = 0
```

```
derecha = len(cursos) - 1
```

MIENTRAS izquierda <= derecha:

```
    medio = (izquierda + derecha) // 2
```

Si cursos[medio].nombre == busqueda:

RETORNAR True

Si cursos[medio].nombre < busqueda:

```
    izquierda = medio + 1 // Buscar en mitad derecha
```

SINO:

```
    derecha = medio - 1 // Buscar en mitad izquierda
```

RETORNAR False

Complejidad: $O(\log n)$ **Mejor caso:** $O(1)$ - elemento en el medio **Peor caso:** $O(\log n)$ -
aproximadamente 10 comparaciones para 1000 elementos **Ventaja:** Muy rápida **Desventaja:** Necesita
lista ordenada

Verificación de orden:

python

```
def verificar_orden_alfabetico(cursos)
```

- Recorre la lista comparando elementos consecutivos
- Retorna False si encuentra desorden
- Complejidad: $O(n)$

Módulo 5: Ordenamiento

Ordenamiento por Burbuja

python

```
def ordenar_nota(cursos)
```

Propósito: Ordenar cursos de mayor a menor nota

Algoritmo:

PARA i de 0 hasta n-1:

 intercambios = 0

 PARA j de 0 hasta n-i-2:

 Si cursos[j].nota < cursos[j+1].nota:

 INTERCAMBIAR cursos[j] con cursos[j+1]

 intercambios += 1

Si intercambios == 0:

 ROMPER // Ya está ordenado

Complejidad:

- Mejor caso: $O(n)$ - lista ya ordenada
- Promedio: $O(n^2)$
- Peor caso: $O(n^2)$ - lista inversa

Ventaja: Simple de entender e implementar **Desventaja:** Lento para listas grandes

Ordenamiento por Inserción

python

```
def ordenar_curso(cursos)
```

Propósito: Ordenar cursos alfabéticamente

Algoritmo:

PARA i de 1 hasta n-1:

 curso_actual = cursos[i]

 j = i - 1

MIENTRAS $j \geq 0$ Y `cursos[j].nombre > curso_actual.nombre`:

`cursos[j+1] = cursos[j]`

`j -= 1`

`cursos[j+1] = curso_actual`

Complejidad:

- Mejor caso: $O(n)$ - lista ya ordenada
- Promedio: $O(n^2)$
- Peor caso: $O(n^2)$

Ventaja: Eficiente para listas pequeñas, estable **Desventaja:** $O(n^2)$ para listas grandes

Módulo 6: Historial (Pila)

python

`def registrar_cambio(historial, tipo_accion, detalles)`

- **Operación:** Simula PUSH en pila
- **Complejidad:** $O(1)$
- **Acción:** `historial.append(cambio)`

python

`def mostrar_historial(historial)`

- **Recorrido:** De fin a inicio (inverso)
- **Complejidad:** $O(n)$
- **Loop:** `for i in range(len(historial)-1, -1, -1)`

python

`def deshacer_ultimo_cambio(historial)`

- **Operación:** Simula POP en pila
- **Complejidad:** $O(1)$
- **Acción:** `historial.pop()`

Módulo 7: Cola de Revisiones (FIFO)

python

```
def agregar_solicitud_revision(cola_revisiones, cursos)
```

- **Operación:** Enqueue (agregar)
- **Complejidad:** $O(1)$
- **Acción:** `cola_revisiones.append(solicitud)`

python

```
def procesar_solicitud_revision(cola_revisiones, cursos)
```

- **Operación:** Dequeue (procesar)
- **Complejidad:** $O(1)$
- **Acción:** `cola_revisiones.popleft()`
- **Opciones:**
 1. Aprobar y cambiar nota
 2. Rechazar
 3. Regresar a la cola

python

```
def ver_cola_revisiones(cola_revisiones)
```

- **Recorrido:** Secuencial FIFO
- **Complejidad:** $O(n)$

Algoritmos Implementados

Resumen de Complejidades

Algoritmo	Complejidad	Caso	Uso
Búsqueda Lineal	$O(n)$	Peor	Listas pequeñas/desordenadas
Búsqueda Binaria	$O(\log n)$	Peor	Listas grandes/ordenadas
Ordenamiento Burbuja	$O(n^2)$	Promedio	Notas (educativo)
Ordenamiento Inserción	$O(n^2)$	Promedio	Nombres (educativo)
Promedio	$O(n)$	-	Cálculo simple
Historial LIFO	$O(1)$	Push/Pop	Registro de cambios
Cola FIFO	$O(1)$	Enqueue/Dequeue	Solicitudes

1. MOSTRAR MENÚ PRINCIPAL

FUNCIÓN MostrarMenu()

IMPRIMIR "===== GESTOR DE NOTAS ACADÉMICAS ====="

IMPRIMIR "1. Registrar nuevo curso y nota"

IMPRIMIR "2. Mostrar todos los cursos y notas"

IMPRIMIR "3. Calcular promedio general"

IMPRIMIR "4. Contar cursos aprobados y reprobados"

IMPRIMIR "5. Buscar curso por nombre (Lineal)"

IMPRIMIR "6. Actualizar nota de un curso"

IMPRIMIR "7. Eliminar un curso"

IMPRIMIR "8. Ordenar cursos por nota"

IMPRIMIR "9. Ordenar cursos por nombre"

IMPRIMIR "10. Buscar curso por nombre (Binaria)"

IMPRIMIR "11. Simular cola de solicitudes de revisión"

IMPRIMIR "12. Mostrar historial de cambios"

IMPRIMIR "13. Salir"

FIN FUNCIÓN